CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT

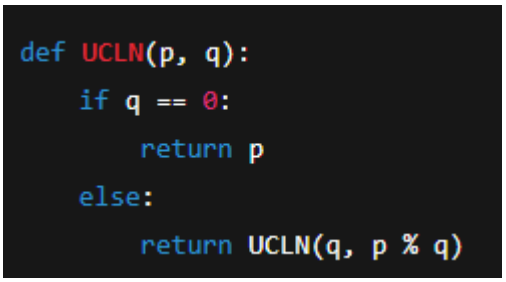
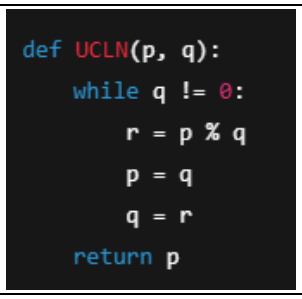
Câu 1 [DAQ]: Giải thuật tính UCLN của hai số nguyên dương p và q (p > q) được mô tả như sau :
Gọi r là số dư trong phép chia q cho p:
+ Nếu r = 0 thì q là UCLN
+ Nếu r khác 0 thì gán p giá trị của q, gán cho q giá trị của r rồi lặp lại quá trình.

a) Xây dựng một định nghĩa đệ quy cho hàm UCLN(p, q)
b) Viết một giải thuật đệ quy và một giải thuật không đệ quy thể hiện hàm đó.

Lời giải

a/ Định nghĩa đệ quy cho hàm UCLN(p, q)
Giải thuật Euclid tìm Ước chung lớn nhất của hai số nguyên dương ppp và qqq (p>qp > qp>q) được mô tả bằng định nghĩa đệ quy như sau: $UCLN(p,q) = \begin{cases} p & (q = 0) \\ UCLN(q, p \mod q) & (q \neq 0) \end{cases}$

b/ Viết một giải thuật đệ quy và một giải thuật không đệ quy thể hiện hàm đó.

Giải thuật đệ quy	Giải thuật không đệ quy
	

```
def UCLN(p, q):
    if q == 0:
        return p
    else:
        return UCLN(q, p % q)
```

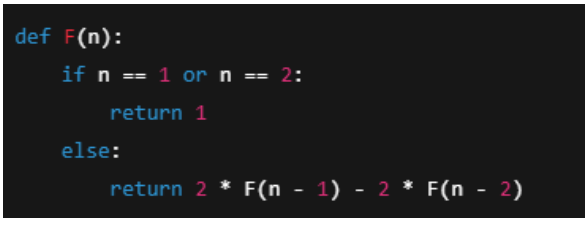
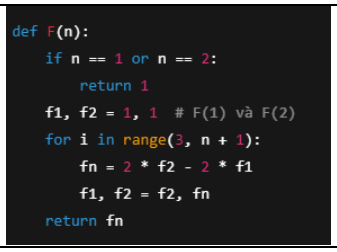
```
def UCLN(p, q):
    while q != 0:
        r = p % q
        p = q
        q = r
    return p
```

*Cách bấm máy casio tìm UCLN(a, b) = ALPHA X (a SHIFT) b) =
*Ví dụ :
Gọi UCLN(48, 18):
48 % 18 = 12 → gọi UCLN(18, 12)
18 % 12 = 6 → gọi UCLN(12, 6)
12 % 6 = 0 → trả về 6
Kết quả: UCLN(48, 18) = 6

Câu 2 [DAQ]: Cho một dãy số nguyên: $F(n) = \begin{cases} 1 & n = 1,2 \\ 2F(n-1) - 2F(n-2) & n > 2 \end{cases}$

a) Viết giải thuật đệ quy xác định giá trị của phần tử thứ n trong dãy.
b) Viết giải thuật không đệ quy xác định giá trị của phần tử thứ n trong dãy.

Lời giải

Giải thuật đệ quy	Giải thuật không đệ quy
	

```
def F(n):
    if n == 1 or n == 2:
        return 1
    else:
        return 2 * F(n - 1) - 2 * F(n - 2)
```

```
def F(n):
    if n == 1 or n == 2:
        return 1
    f1, f2 = 1, 1  # F(1) và F(2)
    for i in range(3, n + 1):
        fn = 2 * f2 - 2 * f1
        f1, f2 = f2, fn
    return fn
```

*Ví dụ :

Tính $F(5)$:

- $F(1) = F(2) = 1$
- $F(3) = 2 \times F(2) - 2 \times F(1) = 2 \times 1 - 2 \times 1 = 0$
- $F(4) = 2 \times F(3) - 2 \times F(2) = 2 \times 0 - 2 \times 1 = -2$
- $F(5) = 2 \times F(4) - 2 \times F(3) = 2 \times (-2) - 2 \times 0 = -4$

Vậy $F(5) = -4$

Câu 3 [DAQ]:

2.4 Xét định nghĩa đệ quy:

$$Acker(m,n) = \begin{cases} n+1 \ \ nếu\ m = 0 \\ Acker(m-1,1)\ nếu\ m > 0 \text{ và } n = 0 \\ Acker(m-1, Acker(m,n-1))\ nếu\ m > 0 \text{ và } n > 0 \end{cases}$$

Hàm này gọi là hàm Ackermann. Nó có đặc điểm là giá trị của nó tăng rất nhanh, ứng với giá trị nguyên của m và n.

a) Hãy xác định Acker (1, 2).

b) Viết một giải thuật đệ quy thực hiện tính giá trị của hàm này.

- **Acker(1, 2)**: Theo trường hợp thứ ba của định nghĩa (m > 0 và n > 0), ta có:

$$Acker(1,2) = Acker(1-1, Acker(1,2-1)) = Acker(0, Acker(1,1))$$

- Bây giờ ta cần tính **Acker(1, 1)**. Áp dụng lại trường hợp thứ ba:

$$Acker(1,1) = Acker(1-1, Acker(1,1-1)) = Acker(0, Acker(1,0))$$

- Tiếp theo, tính **Acker(1, 0)**. Theo trường hợp thứ hai (m > 0 và n = 0):

$$Acker(1,0) = Acker(1-1,1) = Acker(0,1)$$

- Bây giờ tính **Acker(0, 1)**. Theo trường hợp thứ nhất (m = 0):

$$Acker(0,1) = 1+1 = 2$$

- Thay giá trị của **Acker(0, 1)** vào biểu thức của **Acker(1, 0)**:

$$Acker(1,0) = Acker(0,1) = 2$$

- Thay giá trị của **Acker(1, 0)** vào biểu thức của **Acker(1, 1)**:

$$Acker(1,1) = Acker(0, Acker(1,0)) = Acker(0,2)$$

- Cuối cùng, tính **Acker(0, 2)**. Theo trường hợp thứ nhất (m = 0):

$$Acker(0,2) = 2+1 = 3$$

- Thay giá trị của **Acker(0, 2)** vào biểu thức của **Acker(1, 1)**:

$$Acker(1,1) = 3$$

- Cuối cùng, thay giá trị của **Acker(1, 1)** vào biểu thức ban đầu của **Acker(1, 2)**:

$$Acker(1,2) = Acker(0, Acker(1,1)) = Acker(0,3)$$

- Và **Acker(0, 3)** theo trường hợp thứ nhất (m = 0):

$$Acker(0,3) = 3+1 = 4$$

Vậy, **Acker(1, 2) = 4.**

b/

Giải thuật	Hàm

Hàm Acker(m, n):
  Nếu m = 0:
  Trả về n + 1
  Nếu m > 0 và n = 0:
  Trả về Acker(m - 1, 1)
  Nếu m > 0 và n > 0:
  Trả về Acker(m - 1, Acker(m, n - 1))
	def ackermann(m, n):
	  if m == 0:
	  return n + 1
	  elif m > 0 and n == 0:
	  return ackermann(m - 1, 1)
	  elif m > 0 and n > 0:
	  return ackermann(m - 1, ackermann(m, n - 1))
	  else:
	  raise ValueError("Tham số phải là số nguyên không âm.")

Câu 4 [DAQ]:

2.5 Thiết kế giải thuật tìm kiếm nhị phân dưới dạng giải thuật đệ quy.

2.6 Viết giải thuật đệ quy tìm tất cả các hoán vị của N số nguyên dương đầu tiên với N được nhập từ bàn phím.

2.5

Giải thuật	Hàm

Thuật toán TimKiemNhiPhan(A, left, right, x)
  Nếu left > right thì
  Trả về -1 // Không tìm thấy
  mid ← (left + right) div 2
  Nếu A[mid] = x thì
  Trả về mid // Tìm thấy vị trí x
  Nếu A[mid] > x thì
  Trả về TimKiemNhiPhan(A, left, mid - 1, x)
  Ngược lại
  Trả về TimKiemNhiPhan(A, mid + 1, right, x)
	def binary_search_recursive(A, left, right, x):
	  if left > right:
	  return -1
	  mid = (left + right) // 2
	  if A[mid] == x:
	  return mid
	  elif A[mid] > x:
	  return binary_search_recursive(A, left, mid - 1, x)
	  else:
	  return binary_search_recursive(A, mid + 1, right, x)

2.6

Giải thuật	Hàm

Thuật toán LietKeHoanVi(a, k, N)
  Nếu k = N thì
  Xuất a[1..N] // Đã có 1 hoán vị hoàn chỉnh
  Khác thì
  Với i từ k đến N làm:
  Hoán đổi a[k] ↔ a[i]
  Gọi đệ quy LietKeHoanVi(a, k + 1, N)
  Hoán đổi lại a[k] ↔ a[i] // Trả mảng về trạng thái ban đầu
 	
Nhập N
Khởi tạo mảng a[1..N] với a[i] = i
Gọi LietKeHoanVi(a, 1, N)
	def hoan_vi(a, k, n):
	  if k == n:
	  print(a)
	  else:
	  for i in range(k, n):
	  a[k], a[i] = a[i], a[k]
	  hoan_vi(a, k + 1, n)
	  a[k], a[i] = a[i], a[k] # backtrack
	
	# Nhập N từ bàn phím
	N = int(input("Nhập N: "))
	a = list(range(1, N + 1))
	hoan_vi(a, 0, N)

Câu 5 [DAQ]: Xét bài toán tìm tổ tiên của loài ong:

Nếu một con ong được sinh ra bởi sự thụ tinh của ong bố và ong mẹ thì nó sẽ là ong cái.

Nếu một con ong được sinh ra bởi một con ong cái thì nó sẽ là ong đực.

Như vậy, nếu xuất phát từ ong đực, thì thế hệ trước của nó sẽ là một ong cái. Nếu xuất phát từ ong cái thì thế hệ trước của nó sẽ là một ong đực và một ong cái.

Xét thế hệ hiện tại F(0) là một con ong cái. F(1) là thế hệ ngay trước sinh ra F(0). Xây dựng công thức đệ quy và tính số ong ở thế hệ thứ n.

```python
def count_bees(n):
    """
    Tính số lượng ong ở thế hệ thứ n theo bài toán tổ tiên của loài ong.

    Args:
        n: Số nguyên không âm, đại diện cho thế hệ (0 là hiện tại).

    Returns:
        Số lượng ong ở thế hệ thứ n.
        Trả về thông báo lỗi nếu n là số âm.
    """
    if not isinstance(n, int):
        return "Lỗi: n phải là một số nguyên."
    if n < 0:
        return "Lỗi: n phải là một số không âm."
    if n == 0:
        return 1
    elif n == 1:
        return 2
    else:
        return count_bees(n - 1) + count_bees(n - 2)

# Ví dụ sử dụng:
n = 5
so_ong = count_bees(n)
print(f"Số ong ở thế hệ thứ {n} là: {so_ong}")

n_am = -2
so_ong_am = count_bees(n_am)
print(so_ong_am)

n_khong_phai_so = "a"
so_ong_khong_phai_so = count_bees(n_khong_phai_so)
print(so_ong_khong_phai_so)
```

Câu 6 [DAQ]: Giả sử cần quản lý một lớp học bao gồm các sinh viên. Mỗi sinh viên gồm các thông tin sau : Họ và tên, lớp, số báo danh, điểm trung bình. Anh/chị hãy :

a) Viết dạng cài đặt danh sách này bằng cấu trúc danh sách liên kết đơn.

b) Với cấu trúc danh sách đã cài đặt, viết các chương trình con thực hiện các yêu cầu như sau:

+ Nhập vào danh sách gồm n sinh viên.

+ Loại bỏ khỏi danh sách những sinh viên có điểm trung bình dưới 5.

+ Sắp xếp danh sách theo trường lớp tăng dần.

+ In ra màn hình danh sách sinh viên theo từng lớp.

Với n là một số nguyên dương tự nhập từ bàn phím.

Lời giải

```python
class SinhVien:
    def __init__(self, ho_ten, lop, so_bao_danh, diem_tb):
        self.ho_ten = ho_ten
        self.lop = lop
        self.so_bao_danh = so_bao_danh
        self.diem_tb = diem_tb
        self.next = None  # Con trỏ đến sinh viên tiếp theo
```

```
class DanhSachSinhVien:
    def __init__(self):
        self.head = None  # Node đầu danh sách

    def them_cuoi(self, sv_moi):
        if self.head is None:
            self.head = sv_moi
        else:
            current = self.head
            while current.next:
                current = current.next
            current.next = sv_moi

    def nhap_danh_sach(self, n):
        for _ in range(n):
            ho_ten = input("Nhập họ và tên: ")
            lop = input("Nhập lớp: ")
            so_bao_danh = int(input("Nhập số báo danh: "))
            diem_tb = float(input("Nhập điểm trung bình: "))
            sv_moi = SinhVien(ho_ten, lop, so_bao_danh, diem_tb)
            self.them_cuoi(sv_moi)

    def xoa_sinh_vien_diem_thap(self):
        current = self.head
        prev = None
        while current:
            if current.diem_tb < 5:
                if prev is None:  # Nếu sinh viên cần xóa là node đầu
                    self.head = current.next
                else:
                    prev.next = current.next
            else:
                prev = current
            current = current.next

    def sap_xep_theo_lop(self):
        if self.head is None or self.head.next is None:
            return
        # Sắp xếp danh sách theo lớp
        current = self.head
        while current:
            next_node = current.next
```

```
            while next_node:
                if current.lop > next_node.lop:
                    # Hoán đổi thông tin sinh viên
                    current.ho_ten, next_node.ho_ten = next_node.ho_ten, current.ho_ten
                    current.lop, next_node.lop = next_node.lop, current.lop
                    current.so_bao_danh, next_node.so_bao_danh = next_node.so_bao_danh,
current.so_bao_danh
                    current.diem_tb, next_node.diem_tb = next_node.diem_tb, current.diem_tb
                next_node = next_node.next
            current = current.next

    def in_danh_sach_theo_lop(self):
        current = self.head
        while current:
            print(f"Lớp {current.lop}:")
            while current and current.lop == current.lop:
                print(f"  Họ tên: {current.ho_ten} | SBD: {current.so_bao_danh} | Điểm TB:
{current.diem_tb:.2f}")
                current = current.next
# Tạo danh sách sinh viên
ds = DanhSachSinhVien()

# Nhập số sinh viên
n = int(input("Nhập số sinh viên: "))
ds.nhap_danh_sach(n)

# Xóa những sinh viên có điểm trung bình dưới 5
ds.xoa_sinh_vien_diem_thap()

# Sắp xếp danh sách theo lớp
ds.sap_xep_theo_lop()

# In ra màn hình danh sách sinh viên theo từng lớp
ds.in_danh_sach_theo_lop()
```

Câu 7 [DAQ]: Xoá phần tử có giá trị là 4 trong danh sách đã tạo nếu không có phần tử nào thì hiện thị không có phần tử nào.

```
class Node:
    def __init__ (self, key):
        self.key = key
        self.next = None

class LL:
```

```
    def __init__ (self):
        self.head = None
        self.count = 0

    def insert(L, k):
        node = Node(k)
        node.next = L.head
        L.head = node

    def delete(L, k):
        found = False
        if L.head != None:
            if L.head.key == k:
                L.head = L.head.next

            else:
                z = y = L.head
                while y != None and y.key != k:
                    z = y
                    y= y.next
                if y != None:
                    z.next = y.next
        if not found:
            print(f"Không có phần tử nào có giá trị là {k}.")
    def show(L):
        x = L.head
        while x != None:
            print(x.key, end = " -> ")
            x = x.next
        print("None")
L = LL()
L.insert(1)
L.insert(1)
L.insert(5)
L.insert(6)
L.insert(2)

print(f"\nXoá phần tử có giá trị là 4")
L.delete(4)

print("\nDanh sách sau khi xoá là:")
L.show()
```

Câu 8 [DAQ]: Một cửa hàng bán quần áo muốn quản lí danh sách các khách hàng thân thiết dựa vào các thông tin: họ tên khách hàng, ngày sinh khách hàng, sđt, tổng đơn mua. Sử dụng cấu trúc dữ liệu danh sách liên kết đơn đã học để giúp cửa hàng thực hiện các công việc sau:

a) thêm vào một khách hàng mới.

b) hiển thị thông tin của khách hàng dựa vào mã khách hàng hoặc họ tên khách hàng.

c) xoá khách hàng có tổng đơn mua nhỏ hơn 1tr.

d) liệt kê thông tin các khách hàng có ngày sinh nhật cụ thể nào đó.

```
class KhachHang:
    def __init__(self, ma_khach_hang, ho_ten, ngay_sinh, sdt, tong_don_mua):
        self.ma_khach_hang = ma_khach_hang
        self.ho_ten = ho_ten
        self.ngay_sinh = ngay_sinh
        self.sdt = sdt
        self.tong_don_mua = tong_don_mua
        self.next = None

class DanhSachKhachHang:
    def __init__(self):
        self.head = None

    def them_khach_hang(self, ma_khach_hang, ho_ten, ngay_sinh, sdt, tong_don_mua):
        khach_hang_moi = KhachHang(ma_khach_hang, ho_ten, ngay_sinh, sdt, tong_don_mua)
        if not self.head:
            self.head = khach_hang_moi
            return
        current = self.head
        while current.next:
            current = current.next
        current.next = khach_hang_moi

    def hien_thi_khach_hang(self, tim_kiem):
        current = self.head
        found = False
        print("Thông tin khách hàng:")
        while current:
            if current.ma_khach_hang == tim_kiem or current.ho_ten.lower() ==
tim_kiem.lower():
                print(f"  Mã KH: {current.ma_khach_hang}")
                print(f"  Họ tên: {current.ho_ten}")
                print(f"  Ngày sinh: {current.ngay_sinh}")
                print(f"  SĐT: {current.sdt}")
                print(f"  Tổng đơn mua: {current.tong_don_mua}")
                found = True
```

```
            current = current.next
        if not found:
            print(f"Không tìm thấy khách hàng với mã/tên: {tim_kiem}")

    def xoa_khach_hang_duoi_1tr(self):
        if not self.head:
            return

        # Xóa các nút ở đầu danh sách
        while self.head and self.head.tong_don_mua < 1000000:
            self.head = self.head.next

        current = self.head
        previous = None
        while current:
            if current.tong_don_mua < 1000000:
                if previous:
                    previous.next = current.next
                # Trường hợp nút cần xóa không phải là head đã được xử lý ở trên
            else:
                previous = current
            current = current.next

    def liet_ke_khach_hang_sinh_nhat(self, ngay_sinh_cu_the):
        current = self.head
        found = False
        print(f"Danh sách khách hàng có ngày sinh nhật {ngay_sinh_cu_the}:")
        while current:
            if current.ngay_sinh == ngay_sinh_cu_the:
                print(f"  Mã KH: {current.ma_khach_hang}")
                print(f"  Họ tên: {current.ho_ten}")
                print(f"  Ngày sinh: {current.ngay_sinh}")
                print(f"  SĐT: {current.sdt}")
                print(f"  Tổng đơn mua: {current.tong_don_mua}")
                found = True
            current = current.next
        if not found:
            print(f"Không có khách hàng nào có ngày sinh nhật {ngay_sinh_cu_the}")

    # Tạo một danh sách khách hàng
danh_sach = DanhSachKhachHang()

# Thêm khách hàng
```

```
danh_sach.them_khach_hang("KH001", "Nguyễn Văn A", "01/01/2000", "0901234567", 1500000)
danh_sach.them_khach_hang("KH002", "Trần Thị B", "15/05/1995", "0987654321", 800000)
danh_sach.them_khach_hang("KH003", "Lê Văn C", "01/01/2002", "0911223344", 2200000)

# Hiển thị thông tin khách hàng
danh_sach.hien_thi_khach_hang("KH002")
danh_sach.hien_thi_khach_hang("Trần Thị B")
danh_sach.hien_thi_khach_hang("Nguyễn Văn D") # Không tồn tại

# Liệt kê khách hàng có sinh nhật 01/01/2000
danh_sach.liet_ke_khach_hang_sinh_nhat("01/01/2000")

# Xóa khách hàng có tổng đơn mua dưới 1 triệu
danh_sach.xoa_khach_hang_duoi_1tr()

# Hiển thị lại danh sách sau khi xóa
print("\nDanh sách khách hàng sau khi xóa:")
current = danh_sach.head
while current:
    print(f"Mã KH: {current.ma_khach_hang}, Tên: {current.ho_ten}, Tổng mua:
{current.tong_don_mua}")
    current = current.next

# Liệt kê lại khách hàng có sinh nhật 01/01
danh_sach.liet_ke_khach_hang_sinh_nhat("01/01/2000")
danh_sach.liet_ke_khach_hang_sinh_nhat("01/01/2002")
```

Câu 9 [DAQ]: Bài 4.9 trang 147 Giáo trình Cấu trúc dữ liệu và giải thuật.

4.9. Giải thuật quản lý sinh viên

Mô tả cấu trúc dữ liệu:

- **Node (Nút):**
  - ho_ten: Kiểu chuỗi (string)
  - lop: Kiểu chuỗi (string)
  - diem_tong_ket: Kiểu số thực (float)
  - next: Con trỏ đến nút tiếp theo
- **Danh sách liên kết đơn:**
  - head: Con trỏ đến nút đầu tiên
  - tail: Con trỏ đến nút cuối cùng (có thể có hoặc không)

Các hàm (phương thức):

a) Nhập danh sách gồm n sinh viên:

1. Hỏi người dùng nhập số lượng sinh viên n.
2. Khởi tạo một danh sách liên kết đơn rỗng.

3. Lặp lại n lần:
    - Hỏi người dùng nhập họ tên, lớp và điểm tổng kết của sinh viên.
    - Tạo một nút mới với thông tin này.
    - Thêm nút mới vào cuối danh sách liên kết.

b) Thêm một sinh viên mới vào vị trí thứ k:

1. Hỏi người dùng nhập vị trí k.
2. Nếu k < 0 hoặc k > độ dài danh sách, thông báo lỗi và kết thúc.
3. Tạo một nút mới với thông tin của sinh viên "Nguyen Van A", "K80C", 8.5.
4. Nếu k = 0:
    - Cho next của nút mới trỏ đến head hiện tại.
    - Cập nhật head thành nút mới.
    - Nếu danh sách rỗng, cập nhật tail thành nút mới.
5. Ngược lại (k > 0):
    - Duyệt danh sách đến vị trí k - 1.
    - Cho next của nút mới trỏ đến nút thứ k.
    - Cho next của nút thứ k - 1 trỏ đến nút mới.
    - Nếu k là vị trí cuối cùng, cập nhật tail thành nút mới.

c) In ra danh sách các học sinh:

1. Nếu danh sách rỗng, in "Danh sách rỗng".
2. Bắt đầu từ nút head.
3. Lặp lại cho đến khi đến cuối danh sách:
    - In thông tin của nút hiện tại.
    - Di chuyển đến nút tiếp theo.

d) Loại bỏ các sinh viên có điểm tổng kết nhỏ hơn 5:

1. Nếu danh sách rỗng, thông báo "Danh sách rỗng" và kết thúc.
2. Duyệt danh sách từ đầu:
    - Nếu nút hiện tại có diem_tong_ket < 5:
        - Nếu nút hiện tại là head:
            - Cập nhật head thành nút tiếp theo.
            - Nếu head trở thành null, cập nhật tail thành null.
        - Ngược lại:
            - Cho next của nút trước trỏ đến next của nút hiện tại.
            - Nếu nút hiện tại là tail, cập nhật tail thành nút trước.
        - Giải phóng bộ nhớ của nút hiện tại.
    - Ngược lại:
        - Di chuyển đến nút tiếp theo.

Hàm:

```
class quan_li:
    def __init__ (self, ho_ten, lop, diem):
        self.ho_ten = ho_ten
        self.lop = lop
        self.diem = diem
        self.next = None
```

```
class sinh_vien:
    def __init__ (self):
        self.head = None
        self.count = 0

    def nhap_danh_sach(self, n):
        for _ in range(n):
            ho_ten = input("Nhập họ và tên: ")
            lop = input("Nhập lớp: ")
            diem = float(input("Nhập điểm tổng kết: "))
            self.them_sv(self.count, ho_ten, lop, diem)

    def them_sv(self, vi_tri, ho_ten, lop, diem):
        if vi_tri < 0:
            print("Vị trí này không hợp lệ")
            return

        newnode = quan_li(ho_ten, lop, diem)

        if vi_tri == 0:
            newnode.next = self.head
            self.head = newnode
            self.count += 1
            return
        current = self.head
        count = 0

        for _ in range (vi_tri - 1):
            current = current.next
            count += 1



        if current is quan_li:
                print("Ngoài danh sách")
                return

        newnode.next = current.next
        current.next = newnode

    def in_ds(self):
        current = self.head
        while current:
```

```
            print(f"Lớp {current.lop}:")
            while current:
                print(f"  Họ tên: {current.ho_ten} | SBD: {current.so_bao_danh} | Điểm:
{current.diem_tb:.2f}")
                current = current.next

    def xoa_sv(self, key):
        current = self.head
        prev = None
        while current is not None:
            if current.diem < key:
                if prev is None:
                    self.head = current.next
                    current = self.head
                else:
                    prev.next = current.next
                    current = prev.next
                self.count -= 1
            else:
                prev = current
                current = current.next

    def hien_thi(self):
        current = self.head
        while current:
            print(f"Họ và tên: {current.ho_ten} | Lớp: {current.lop} | Điểm: {current.diem}")
            current = current.next
        print("None")

L = sinh_vien()
n = int(input("Nhập giá trị n = "))
L.nhap_danh_sach(n)

k = int(input("Nhập vị trí k:"))
print("Vị trí chèn:", k)
L.them_sv(k, "Nguyen Van A", "K80C", 8.5)
L.hien_thi()

L.xoa_sv(5)
L.hien_thi()
```

Câu 10 [DAQ]: Bài 4.10 trang 147 Giáo trình Cấu trúc dữ liệu và giải thuật.

Ý tưởng:

- Mỗi con vượn được lưu trong một nút của danh sách liên kết đơn.

- Mỗi nút chứa thông tin: Mã ID, Tên, Chiều cao, Cân nặng, Tuổi.
- Danh sách liên kết được tạo bằng cách nhập từ bàn phím.
- Thực hiện các thao tác:
    - Nhập danh sách n con vượn.
    - Thêm 1 con vượn mới vào vị trí đầu danh sách.
    - Lọc và in ra các con vượn có tuổi > 22.
    - Hiển thị danh sách các con vượn theo thứ tự giảm dần của cân nặng (có thể bằng cách sắp xếp danh sách).

Hàm:

```
class cong_vien:
    def __init__ (self, ID, ten, chieu_cao, can_nang, tuoi):
        self.ID = ID
        self.ten = ten
        self.chieu_cao = chieu_cao
        self.can_nang = can_nang
        self.tuoi = tuoi
        self.next = None

class vuon:
    def __init__ (self):
        self.head = None
        self.count = 0

    def nhap_danh_sach(self, n):
        for _ in range(n):
            ID = input("Nhập ID: ")
            ten = input("Nhập tên: ")
            chieu_cao = int((input("Nhập chiều cao: ")))
            can_nang = int(input("Nhập cân nặng: "))
            tuoi = int(input("Nhập tuổi: "))
            self.thong_tin_moi(self.count, ID, ten, chieu_cao, can_nang, tuoi)

    def thong_tin_moi(self, pos, ID, ten, chieu_cao, can_nang, tuoi):
        newnode = cong_vien(ID, ten, chieu_cao, can_nang, tuoi)
        if pos == 0:
            newnode.next = self.head
            self.head = newnode
        else:
            pPre = self.head
            count = 0
            while pPre is not None and count < pos - 1:
                pPre = pPre.next
                count += 1
```

```
            if pPre is None:
                print("Vị trí chèn không hợp lệ!")
            else:
                newnode.next = pPre.next
                pPre.next = newnode
        self.count += 1

    def tuoi_22(self):
        cur = self.head
        while cur:
            if cur.tuoi > 22:
                return cur
            cur = cur.next
        return None

    def in_ds(self):
        current = self.head
        while current:
            print(f"ID: {current.ID} | Tên: {current.ten} | Tuổi: {current.tuoi} | Chiều cao:
{current.chieu_cao} | Cân nặng: {current.can_nang}")
            current = current.next

    def hien_thi(self):
        current = self.head
        while current:
            print(f"ID: {current.ID} | Tên: {current.ten} | Tuổi: {current.tuoi} | Chiều cao:
{current.chieu_cao} | Cân nặng: {current.can_nang}")
            current = current.next
        print("None")

    def sap_xep_giam_dan(self, theo_truong="can_nang"):
        sorted_head = None
        current = self.head

        while current:
            next_node = current.next
            current.next = None

            if (sorted_head is None or int(getattr(current, theo_truong)) >
int(getattr(sorted_head, theo_truong))):
                current.next = sorted_head
                sorted_head = current
```

```
            else:
                prev = sorted_head
                while (prev.next is not None and int(getattr(prev.next, theo_truong)) >=
int(getattr(current, theo_truong))):
                    prev = prev.next

                current.next = prev.next
                prev.next = current

            current = next_node

        self.head = sorted_head

L = vuon()
n = int(input("Nhập giá trị n = "))
L.nhap_danh_sach(n)

print("\nDanh sách đã nhập:")
L.hien_thi()

print("\n--- Thêm một người mới vào đầu danh sách ---")
ID = int(input("Nhập ID: "))
ten = input("Nhập tên: ")
chieu_cao = int(input("Nhập chiều cao: "))
can_nang = int(input("Nhập cân nặng: "))
tuoi = int(input("Nhập tuổi: "))

L.thong_tin_moi(0, ID, ten, chieu_cao, can_nang, tuoi)

print("\nDanh sách sau khi thêm vào đầu:")
L.hien_thi()

nguoi_22 = L.tuoi_22()
if nguoi_22:
    print(f"\nNgười có tuổi 22: Tên: {nguoi_22.ten}, ID: {nguoi_22.ID}, Chiều cao:
{nguoi_22.chieu_cao}, Cân nặng: {nguoi_22.can_nang}, Tuổi: {nguoi_22.tuoi}")
else:
    print("\nKhông có ai tuổi 22.")

L.sap_xep_giam_dan("can_nang")
print("Danh sách sau khi sắp xếp giảm dần theo cân nặng:")
L.hien_thi()
```

Câu 11 [DAQ]: Bài 4.11 trang 147 Giáo trình Cấu trúc dữ liệu và giải thuật.

Ý tưởng và giải thuật

a) Nhập n (n > 0) và nhập danh sách gồm n đồ ăn đã bán từ bàn phím.

- **Ý tưởng**: Dùng một danh sách liên kết đơn để lưu từng đồ ăn, mỗi nút chứa các thông tin: Mã đồ ăn, Tên đồ ăn, Đơn giá, Số lượng.
- **Giải thuật**:
    1. Nhập số lượng phần tử n.
    2. Với i từ 1 đến n:
        - Nhập Mã đồ ăn, Tên đồ ăn, Đơn giá, Số lượng.
        - Tạo nút mới và thêm vào cuối danh sách liên kết.

b) Thực hiện bán đồ ăn cho khách, với đồ ăn được bán sẽ bổ sung vào vị trí đầu của danh sách và thông tin đồ ăn do người dùng tự nhập.

- **Ý tưởng**: Mỗi lần bán đồ ăn, tạo một nút mới chứa thông tin đồ ăn và thêm vào đầu danh sách.
- **Giải thuật**:
    1. Nhập thông tin đồ ăn được bán (Mã, Tên, Đơn giá, Số lượng).
    2. Tạo nút mới chứa thông tin đó.
    3. Thêm nút vào đầu danh sách.

c) Đưa ra thông tin đồ ăn được bán với Số lượng nhiều nhất.

- **Ý tưởng**: Duyệt danh sách, tìm nút có Số lượng lớn nhất.
- **Giải thuật**:
    1. Khởi tạo biến max = Số lượng của nút đầu tiên.
    2. Duyệt từng nút trong danh sách:
        - Nếu Số lượng của nút > max, cập nhật max và lưu lại nút đó.
    3. In thông tin nút có Số lượng lớn nhất.

d) Tính tổng doanh thu đã bán dựa trên Đơn giá và Số lượng.

- **Ý tưởng**: Duyệt danh sách, tính tổng Đơn giá × Số lượng.
- **Giải thuật**:
    1. Khởi tạo biến tổng doanh thu = 0.
    2. Duyệt từng nút trong danh sách:
        - Cộng Đơn giá × Số lượng vào tổng doanh thu.
    3. In tổng doanh thu.

Hàm:

```
class MonDoAn:
    def __init__(self, ma_do_an, ten_do_an, don_gia, so_luong):
        self.ma_do_an = ma_do_an
        self.ten_do_an = ten_do_an
        self.don_gia = don_gia
        self.so_luong = so_luong
        self.next = None

class DanhSachDoAn:
    def __init__(self):
        self.head = None
```

```
    def nhap_danh_sach(self, n):
        if n <= 0:
            print("Số lượng món ăn phải lớn hơn 0.")
            return

        for _ in range(n):
            ma = input("Nhập Mã đồ ăn: ")
            ten = input("Nhập Tên đồ ăn: ")
            try:
                don_gia = float(input("Nhập Đơn giá: "))
                so_luong = int(input("Nhập Số lượng: "))
                if don_gia < 0 or so_luong < 0:
                    print("Đơn giá và Số lượng không được âm.")
                    continue
            except ValueError:
                print("Định dạng Đơn giá hoặc Số lượng không hợp lệ.")
                continue

            mon_moi = MonDoAn(ma, ten, don_gia, so_luong)
            if not self.head:
                self.head = mon_moi
            else:
                current = self.head
                while current.next:
                    current = current.next
                current.next = mon_moi
        print("Đã nhập xong danh sách đồ ăn.")

    def ban_do_an(self):
        ma = input("Nhập Mã đồ ăn vừa bán: ")
        ten = input("Nhập Tên đồ ăn vừa bán: ")
        try:
            don_gia = float(input("Nhập Đơn giá: "))
            so_luong = int(input("Nhập Số lượng: "))
            if don_gia < 0 or so_luong < 0:
                print("Đơn giá và Số lượng không được âm.")
                return
        except ValueError:
            print("Định dạng Đơn giá hoặc Số lượng không hợp lệ.")
            return

        mon_moi = MonDoAn(ma, ten, don_gia, so_luong)
```

```
        mon_moi.next = self.head
        self.head = mon_moi
        print("Đã thêm thông tin món ăn vừa bán vào đầu danh sách.")

    def tim_mon_ban_nhieu_nhat(self):
        if not self.head:
            print("Danh sách đồ ăn rỗng.")
            return None

        max_so_luong = -1
        mon_nhieu_nhat = None
        current = self.head

        while current:
            if current.so_luong > max_so_luong:
                max_so_luong = current.so_luong
                mon_nhieu_nhat = current
            current = current.next

        if mon_nhieu_nhat:
            print("Món đồ ăn bán với số lượng nhiều nhất:")
            print(f"  Mã đồ ăn: {mon_nhieu_nhat.ma_do_an}")
            print(f"  Tên đồ ăn: {mon_nhieu_nhat.ten_do_an}")
            print(f"  Đơn giá: {mon_nhieu_nhat.don_gia}")
            print(f"  Số lượng: {mon_nhieu_nhat.so_luong}")
            return mon_nhieu_nhat
        else:
            print("Không có thông tin đồ ăn trong danh sách.")
            return None

    def tinh_tong_doanh_thu(self):
        tong_doanh_thu = 0
        current = self.head

        while current:
            tong_doanh_thu += current.don_gia * current.so_luong
            current = current.next

        print(f"Tổng doanh thu đã bán của cửa hàng là: {tong_doanh_thu}")
        return tong_doanh_thu
```

Câu 12 [DAQ]: Bài 4.12 trang 147 Giáo trình Cấu trúc dữ liệu và giải thuật.

Giải thuật:

a) Nhập N (N0) và nhập danh sách gồm N cán bộ từ bàn phím.

Giải thuật:

1. Nhập số lượng cán bộ N từ người dùng. Đảm bảo N0.
2. Khởi tạo một danh sách liên kết đơn rỗng (danh_sach = DanhSachCanBo()).
3. Lặp lại N lần:
    - Nhập Mã cán bộ, Tên cán bộ và Lương từ người dùng.
    - Tạo một đối tượng CanBo mới với thông tin vừa nhập.
    - Thêm đối tượng này vào cuối danh sách liên kết.

b) Hãy thêm một cán bộ mới vào vị trí đầu của danh sách. Thông tin của cán bộ mới do người dùng nhập vào từ bàn phím.

Giải thuật:

1. Nhập Mã cán bộ, Tên cán bộ và Lương của cán bộ mới từ người dùng.
2. Tạo một đối tượng CanBo mới với thông tin vừa nhập.
3. Thêm đối tượng này vào đầu danh sách liên kết bằng cách:
    - Đặt next của đối tượng mới trỏ đến head hiện tại của danh sách.
    - Cập nhật head của danh sách thành đối tượng mới.

c) Sắp xếp danh sách cán bộ theo thứ tự tăng dần của Lương.

Giải thuật:

1. Kiểm tra nếu danh sách rỗng hoặc chỉ có một nút thì không cần sắp xếp.
2. Sử dụng thuật toán sắp xếp nổi bọt (Bubble Sort) hoặc sắp xếp chèn (Insertion Sort) cho danh sách liên kết. Dưới đây là giải thuật sắp xếp nổi bọt:
    - Duyệt qua danh sách nhiều lần.
    - Trong mỗi lần duyệt, so sánh lương của hai nút liền kề.
    - Nếu lương của nút hiện tại lớn hơn lương của nút kế tiếp, hoán đổi vị trí của hai nút đó.
    - Lặp lại cho đến khi không còn cặp nút nào cần hoán đổi.

d) Hiển thị danh sách cán bộ trước và sau khi sắp xếp.

Giải thuật:

1. Tạo một hàm để in danh sách cán bộ (duyệt qua danh sách và in thông tin của mỗi nút).
2. Gọi hàm in danh sách trước khi gọi hàm sắp xếp.
3. Gọi hàm sắp xếp theo lương.
4. Gọi hàm in danh sách sau khi sắp xếp.

e) In ra danh sách cán bộ có mức lương hoặc bằng 1.600.

Giải thuật:

1. Duyệt qua từng nút trong danh sách liên kết.
2. Kiểm tra nếu luong của nút hiện tại bằng 1600.
3. Nếu đúng, in ra thông tin của cán bộ đó.

f) Nhập mã cán bộ của công ty B, được biết danh sách cán bộ công ty B cũng được lưu trữ trên một danh sách móc nối tương tự ở công ty A.

Giải thuật:

1. **Giả định:** Chúng ta có một danh sách liên kết đơn khác (danh_sach_b) đại diện cho danh sách cán bộ của công ty B (cùng cấu trúc CanBo và DanhSachCanBo).
2. Nhập mã cán bộ cần tìm (ma_can_bo_b) từ người dùng.
3. Duyệt qua danh sách cán bộ của công ty A (danh_sach).
4. Trong quá trình duyệt, so sánh ma_can_bo của mỗi cán bộ trong danh_sach với ma_can_bo_b.

5. Nếu tìm thấy cán bộ có mã trùng khớp, in ra thông tin của cán bộ đó từ danh sách của công ty A.

Hàm:

```
class CanBo:
    def __init__(self, ma_can_bo, ten_can_bo, luong):
        self.ma_can_bo = ma_can_bo
        self.ten_can_bo = ten_can_bo
        self.luong = luong
        self.next = None

class DanhSachCanBo:
    def __init__(self):
        self.head = None

    def nhap_danh_sach(self, n):
        if n <= 0:
            print("Số lượng cán bộ phải lớn hơn 0.")
            return

        for _ in range(n):
            ma = input("Nhập Mã cán bộ: ")
            ten = input("Nhập Tên cán bộ: ")
            try:
                luong = float(input("Nhập Lương: "))
                if luong < 0:
                    print("Lương không được âm.")
                    continue
            except ValueError:
                print("Định dạng Lương không hợp lệ.")
                continue

            can_bo_moi = CanBo(ma, ten, luong)
            if not self.head:
                self.head = can_bo_moi
            else:
                current = self.head
                while current.next:
                    current = current.next
                current.next = can_bo_moi
        print("Đã nhập xong danh sách cán bộ.")

    def them_can_bo_dau(self):
        ma = input("Nhập Mã cán bộ mới: ")
        ten = input("Nhập Tên cán bộ mới: ")
```

```
        try:
            luong = float(input("Nhập Lương cán bộ mới: "))
            if luong < 0:
                print("Lương không được âm.")
                return
        except ValueError:
            print("Định dạng Lương không hợp lệ.")
            return

        can_bo_moi = CanBo(ma, ten, luong)
        can_bo_moi.next = self.head
        self.head = can_bo_moi
        print("Đã thêm cán bộ mới vào đầu danh sách.")

    def sap_xep_theo_luong(self):
        if not self.head or not self.head.next:
            return

        sorted = False
        while not sorted:
            sorted = True
            current = self.head
            while current.next:
                if current.luong > current.next.luong:
                    current.luong, current.next.luong = current.next.luong, current.luong
                    current.ma_can_bo, current.next.ma_can_bo = current.next.ma_can_bo,
current.ma_can_bo
                    current.ten_can_bo, current.next.ten_can_bo = current.next.ten_can_bo,
current.ten_can_bo
                    sorted = False
                current = current.next
        print("Đã sắp xếp danh sách cán bộ theo lương tăng dần.")

    def in_danh_sach(self, title="Danh sách cán bộ:"):
        print(title)
        current = self.head
        if not current:
            print("Danh sách rỗng.")
            return
        while current:
            print(f"  Mã: {current.ma_can_bo}, Tên: {current.ten_can_bo}, Lương:
{current.luong}")
            current = current.next
```

```
# Sử dụng các hàm:
danh_sach = DanhSachCanBo()
n = int(input("Nhập số lượng cán bộ ban đầu: "))
danh_sach.nhap_danh_sach(n)

print("\nDanh sách cán bộ trước khi sắp xếp:")
danh_sach.in_danh_sach()

danh_sach.sap_xep_theo_luong()

print("\nDanh sách cán bộ sau khi sắp xếp theo lương:")
danh_sach.in_danh_sach()

def in_can_bo_luong_1600(self):
        print("\nDanh sách cán bộ có lương bằng 1600:")
        found = False
        current = self.head
        while current:
            if current.luong == 1600:
                print(f"  Mã: {current.ma_can_bo}, Tên: {current.ten_can_bo}, Lương:
{current.luong}")
                found = True
            current = current.next
        if not found:
            print("Không có cán bộ nào có lương bằng 1600.")

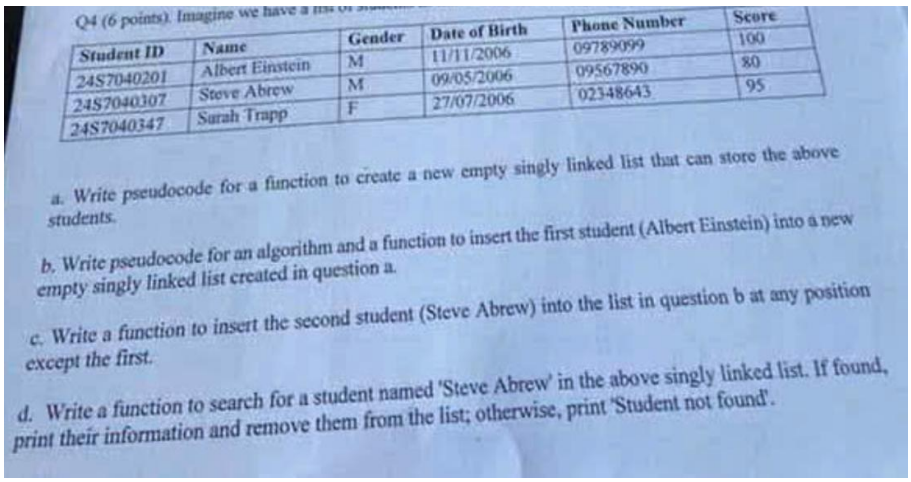
def tim_can_bo_tu_danh_sach_b(self, danh_sach_b):
        ma_can_bo_b = input("Nhập mã cán bộ của công ty B cần tìm trong danh sách công ty A:
")
        found = False
        current_a = self.head
        while current_a:
            if current_a.ma_can_bo == ma_can_bo_b:
                print("\nThông tin cán bộ tìm thấy trong danh sách công ty A:")
                print(f"  Mã: {current_a.ma_can_bo}, Tên: {current_a.ten_can_bo}, Lương:
{current_a.luong}")
                found = True
                break
            current_a = current_a.next

        if not found:
```

```
            print(f"Không tìm thấy cán bộ có mã '{ma_can_bo_b}' từ công ty B trong danh sách công ty A.")

# Ví dụ sử dụng (cần tạo và nhập dữ liệu cho danh_sach_b):
# danh_sach_b = DanhSachCanBo()
# ... (nhập dữ liệu cho danh_sach_b) ...
# danh_sach_a = DanhSachCanBo()
# n_a = int(input("Nhập số lượng cán bộ công ty A: "))
# danh_sach_a.nhap_danh_sach(n_a)
# danh_sach_a.tim_can_bo_tu_danh_sach_b(danh_sach_b)
```

Câu 13 [DAQ]: Đề thứ 2



Q4 (6 points): Imagine we have a list of students:

Student ID	Name	Gender	Date of Birth	Phone Number	Score
24S7040201	Albert Einstein	M	11/11/2006	09789099	100
24S7040307	Steve Abrew	M	09/05/2006	09567890	80
24S7040347	Surah Trapp	F	27/07/2006	02348643	95

a. Write pseudocode for a function to create a new empty singly linked list that can store the above students.

b. Write pseudocode for an algorithm and a function to insert the first student (Albert Einstein) into a new empty singly linked list created in question a.

c. Write a function to insert the second student (Steve Abrew) into the list in question b at any position except the first.

d. Write a function to search for a student named 'Steve Abrew' in the above singly linked list. If found, print their information and remove them from the list; otherwise, print 'Student not found'.

a. Viết mã giả cho một hàm để tạo một danh sách liên kết đơn rỗng mới có thể lưu trữ thông tin sinh viên ở trên.

Mã giả:

```
Hàm TaoDanhSachSinhVienRong():
  // Tạo một nút đầu (head) mới và gán giá trị là NULL (hoặc None)
  dau <- NULL
  // Trả về nút đầu của danh sách rỗng
  trả về dau
```

b. Viết mã giả cho một thuật toán và một hàm để chèn sinh viên đầu tiên (Albert Einstein) vào một danh sách liên kết đơn rỗng mới được tạo ở câu a.

Mã giả cho Thuật toán:

1. Gọi hàm TaoDanhSachSinhVienRong() để tạo một danh sách liên kết đơn rỗng. Gán kết quả cho biến danhSach.
2. Tạo một nút mới để lưu trữ thông tin của sinh viên Albert Einstein.
3. Gán các giá trị sau cho nút mới:
    - maSinhVien = "24S7040201"
    - ten = "Albert Einstein"
    - gioiTinh = "M"
    - ngaySinh = "11/11/2006"

- soDienThoai = "09789099"
- diem = 100
- tiepTheo = NULL

4. Đặt dau của danhSach trỏ đến nút mới này.

Mã giả cho Hàm:

```
Hàm ChenSinhVienDau(dau):
  // Tạo một nút mới
  nutMoi <- TaoNut()
  // Gán thông tin cho nút mới
  nutMoi.maSinhVien <- "24S7040201"
  nutMoi.ten <- "Albert Einstein"
  nutMoi.gioiTinh <- "M"
  nutMoi.ngaySinh <- "11/11/2006"
  nutMoi.soDienThoai <- "09789099"
  nutMoi.diem <- 100
  nutMoi.tiepTheo <- NULL
  // Đặt nút mới làm đầu danh sách
  dau <- nutMoi
  // Trả về đầu của danh sách
  trả về dau

// Giả sử chúng ta đã gọi hàm TaoDanhSachSinhVienRong() và có biến danhSach
danhSach <- ChenSinhVienDau(danhSach)
```

c. Viết một hàm để chèn sinh viên thứ hai (Steve Abrew) vào danh sách ở câu b tại bất kỳ vị trí nào ngoại trừ vị trí đầu tiên.

Mã giả cho Hàm:

```
Hàm ChenSinhVien(dau, viTriChen):
  // Tạo một nút mới cho Steve Abrew
  nutMoi <- TaoNut()
  nutMoi.maSinhVien <- "24S7040307"
  nutMoi.ten <- "Steve Abrew"
  nutMoi.gioiTinh <- "M"
  nutMoi.ngaySinh <- "09/05/2006"
  nutMoi.soDienThoai <- "09567890"
  nutMoi.diem <- 80
  nutMoi.tiepTheo <- NULL

  // Nếu vị trí chèn là 1, không thực hiện (loại trừ vị trí đầu tiên)
  nếu viTriChen bằng 1:
    in "Không thể chèn vào vị trí đầu tiên bằng hàm này."
    trả về dau

  // Duyệt đến vị trí trước vị trí cần chèn
  current <- dau
  dem <- 1
  trong khi current không phải NULL và dem < viTriChen - 1:
    current <- current.tiepTheo
    dem <- dem + 1

  // Nếu tìm thấy vị trí hợp lệ
  nếu current không phải NULL:
    nutMoi.tiepTheo <- current.tiepTheo
    current.tiepTheo <- nutMoi
  khác:
    in "Vị trí chèn không hợp lệ."

  trả về dau

// Ví dụ sử dụng (giả sử danhSach đã có Albert Einstein):
danhSach <- ChenSinhVien(danhSach, 2) // Chèn Steve Abrew vào vị trí thứ 2
danhSach <- ChenSinhVien(danhSach, 3) // Chèn Steve Abrew vào vị trí thứ 3 (nếu dar
```

d. Viết một hàm để tìm kiếm một sinh viên có tên 'Steve Abrew' trong danh sách liên kết đơn ở trên. Nếu tìm thấy, in thông tin của họ và xóa họ khỏi danh sách; ngược lại, in 'Không tìm thấy sinh viên'.

Mã giả cho Hàm:

```
Hàm TimVaXoaSinhVien(dau, tenTimKiem):
  current <- dau
  previous <- NULL
  timThay <- Sai

  // Tìm kiếm sinh viên
  trong khi current không phải NULL:
    nếu current.ten bằng tenTimKiem:
      timThay <- Đúng
      in "Đã tìm thấy sinh viên:"
      in "  Mã SV:", current.maSinhVien
      in "  Tên:", current.ten
      in "  Giới tính:", current.gioiTinh
      in "  Ngày sinh:", current.ngaySinh
      in "  SĐT:", current.soDienThoai
      in "  Điểm:", current.diem
      // Xóa sinh viên
      nếu previous bằng NULL: // Nếu là nút đầu
        dau <- current.tiepTheo
      khác:
        previous.tiepTheo <- current.tiepTheo
      thoát vòng lặp
    khác:
      previous <- current
      current <- current.tiepTheo

  // Nếu không tìm thấy
  nếu không timThay:
    in "Không tìm thấy sinh viên"

  trả về dau

// Ví dụ sử dụng:
danhSach <- TimVaXoaSinhVien(danhSach, "Steve Abrew")
```

Hàm:

```python
class SinhVien:
    def __init__(self, ma_sinh_vien, ten, gioi_tinh, ngay_sinh, so_dien_thoai, diem):
        self.ma_sinh_vien = ma_sinh_vien
        self.ten = ten
        self.gioi_tinh = gioi_tinh
        self.ngay_sinh = ngay_sinh
        self.so_dien_thoai = so_dien_thoai
        self.diem = diem
        self.tiep_theo = None

class DanhSachSinhVien:
    def __init__(self):
        self.dau = None
```

```
    def tao_danh_sach_rong(self):
        """Tạo một danh sách liên kết đơn rỗng."""
        self.dau = None

    def chen_sinh_vien_dau(self):
        """Chèn sinh viên đầu tiên (Albert Einstein) vào danh sách."""
        nut_moi = SinhVien("24S7040201", "Albert Einstein", "M", "11/11/2006", "09789099",
100)
        nut_moi.tiep_theo = self.dau
        self.dau = nut_moi
        print("Đã chèn sinh viên Albert Einstein vào đầu danh sách.")

    def chen_sinh_vien(self, vi_tri_chen):
        """Chèn sinh viên thứ hai (Steve Abrew) vào danh sách tại vị trí cho trước (trừ vị
trí đầu)."""
        if vi_tri_chen == 1:
            print("Không thể chèn vào vị trí đầu tiên bằng hàm này.")
            return

        nut_moi = SinhVien("24S7040307", "Steve Abrew", "M", "09/05/2006", "09567890", 80)

        if not self.dau:
            print("Danh sách rỗng, không thể chèn tại vị trí khác 1.")
            return

        current = self.dau
        dem = 1
        while current and dem < vi_tri_chen - 1:
            current = current.tiep_theo
            dem += 1

        if current:
            nut_moi.tiep_theo = current.tiep_theo
            current.tiep_theo = nut_moi
            print(f"Đã chèn sinh viên Steve Abrew vào vị trí {vi_tri_chen}.")
        else:
            print(f"Vị trí chèn {vi_tri_chen} không hợp lệ.")

    def tim_va_xoa_sinh_vien(self, ten_tim_kiem):
        """Tìm kiếm sinh viên theo tên, in thông tin và xóa nếu tìm thấy."""
        current = self.dau
        previous = None
```

```
        tim_thay = False

        while current:
            if current.ten.lower() == ten_tim_kiem.lower():
                tim_thay = True
                print("Đã tìm thấy sinh viên:")
                print(f"  Mã SV: {current.ma_sinh_vien}")
                print(f"  Tên: {current.ten}")
                print(f"  Giới tính: {current.gioi_tinh}")
                print(f"  Ngày sinh: {current.ngay_sinh}")
                print(f"  SĐT: {current.so_dien_thoai}")
                print(f"  Điểm: {current.diem}")

                if previous is None:
                    self.dau = current.tiep_theo
                else:
                    previous.tiep_theo = current.tiep_theo
                print("Đã xóa sinh viên khỏi danh sách.")
                break
            previous = current
            current = current.tiep_theo

        if not tim_thay:
            print("Không tìm thấy sinh viên")

    def in_danh_sach(self):
        """In thông tin của tất cả sinh viên trong danh sách."""
        current = self.dau
        if not current:
            print("Danh sách sinh viên rỗng.")
            return
        print("Danh sách sinh viên:")
        while current:
            print(f"  Mã SV: {current.ma_sinh_vien}, Tên: {current.ten}, Giới tính:
{current.gioi_tinh}, Ngày sinh: {current.ngay_sinh}, SĐT: {current.so_dien_thoai}, Điểm:
{current.diem}")
            current = current.tiep_theo

# Sử dụng các hàm:
danh_sach = DanhSachSinhVien()

# a) Tạo danh sách rỗng
danh_sach.tao_danh_sach_rong()
```

```
print("Đã tạo danh sách sinh viên rỗng.")

# b) Chèn Albert Einstein
danh_sach.chen_sinh_vien_dau()
danh_sach.in_danh_sach()

# c) Chèn Steve Abrew vào vị trí thứ 2
danh_sach.chen_sinh_vien(2)
danh_sach.in_danh_sach()

# d) Tìm và xóa Steve Abrew
danh_sach.tim_va_xoa_sinh_vien("Steve Abrew")
danh_sach.in_danh_sach()

# Thử tìm một sinh viên không tồn tại
danh_sach.tim_va_xoa_sinh_vien("Sarah Trapp")
danh_sach.in_danh_sach()

# Thêm lại Steve Abrew để thử xóa
danh_sach.chen_sinh_vien(2)
danh_sach.in_danh_sach()
danh_sach.tim_va_xoa_sinh_vien("steve abrew") # Kiểm tra không phân biệt chữ hoa chữ thường
danh_sach.in_danh_sach()
```

Câu 14 [DAQ]:

🔹 1. Tính tổng chữ số của số nguyên dương N

Đề bài: Viết hàm đệ quy tính tổng các chữ số của số nguyên N.
Gợi ý: N % 10 + TongChuSo(N // 10)

🔹 2. Tính giai thừa của N

Đề bài: Viết hàm đệ quy tính giai thừa của N.
Gợi ý:

- n! = n * (n - 1)!
- Điều kiện dừng: 0! = 1

🔹 3. Tính số Fibonacci thứ N

Đề bài: Viết hàm đệ quy tính số Fibonacci tại vị trí thứ N (với F(0)=0, F(1)=1).
Gợi ý:

- F(n) = F(n - 1) + F(n - 2)
- Điều kiện dừng: F(0)=0, F(1)=1

🔹 4. Tính lũy thừa: A^B

Đề bài: Viết hàm đệ quy để tính A^B (A mũ B, với A và B là số nguyên dương).
Gợi ý:

- A^0 = 1
- A^B = A * A^(B-1)

🔹 5. Đảo ngược một số nguyên N

Đề bài: Viết hàm đệ quy để đảo ngược các chữ số của N.
Ví dụ: 1234 → 4321
Gợi ý: Dùng đệ quy và truyền thêm tham số tích lũy kết quả đang xây dựng.

🔹 6. In các số từ 1 đến N (tăng dần và giảm dần)

Đề bài: Viết hàm đệ quy để:

- In từ 1 đến N theo thứ tự tăng
- In từ N về 1 theo thứ tự giảm
  Gợi ý: Gọi hàm với n - 1, rồi in n.

🔹 7. Kiểm tra chuỗi đối xứng (Palindrome)

Đề bài: Viết hàm đệ quy kiểm tra xem một chuỗi có là Palindrome không (chuỗi đọc xuôi và ngược như nhau).
Gợi ý:

- So sánh ký tự đầu và cuối.
- Đệ quy phần giữa.

🔹 8. Liệt kê tất cả các hoán vị của N số (1 đến N)

Đề bài: Viết hàm đệ quy liệt kê tất cả hoán vị của dãy [1, 2, ..., N].
Gợi ý: Dùng hoán đổi + backtracking.

🔹 9. Tính tổng mảng bằng đệ quy

Đề bài: Viết hàm đệ quy tính tổng các phần tử trong mảng a[0..n-1].
Gợi ý:

- Tổng n phần tử: a[n-1] + Tong(a, n-1)

🔹 10. Đổi số nguyên sang hệ nhị phân bằng đệ quy

Đề bài: Viết hàm đệ quy in ra dạng nhị phân của một số nguyên dương N.
Gợi ý:

- N == 0: dừng
- N % 2 là chữ số cuối
- In trước là Binary(N // 2)

```
# 1. Tổng các chữ số của N
def tong_chu_so(n):
    if n < 10:
        return n
```

```python
    return n % 10 + tong_chu_so(n // 10)

# 2. Giai thừa của N
def giai_thua(n):
    if n == 0:
        return 1
    return n * giai_thua(n - 1)


# 3. Số Fibonacci thứ N
def fibonacci(n):
    if n == 0 or n == 1:
        return n
    return fibonacci(n - 1) + fibonacci(n - 2)


# 4. Lũy thừa A^B
def luy_thua(a, b):
    if b == 0:
        return 1
    return a * luy_thua(a, b - 1)


# 5. Đảo ngược số nguyên N
def dao_nguoc_so(n, ket_qua=0):
    if n == 0:
        return ket_qua
    return dao_nguoc_so(n // 10, ket_qua * 10 + n % 10)


# 6. In từ 1 đến N (tăng dần)
def in_tang(n):
    if n == 0:
        return
    in_tang(n - 1)
    print(n, end=' ')

# In từ N về 1 (giảm dần)
def in_giam(n):
    if n == 0:
        return
    print(n, end=' ')
    in_giam(n - 1)
```

```python
# 7. Kiểm tra chuỗi Palindrome
def is_palindrome(s):
    if len(s) <= 1:
        return True
    if s[0] != s[-1]:
        return False
    return is_palindrome(s[1:-1])


# 8. Hoán vị N số từ 1 đến N
def hoan_vi(a, k, n):
    if k == n:
        print(a)
    else:
        for i in range(k, n):
            a[k], a[i] = a[i], a[k]
            hoan_vi(a, k + 1, n)
            a[k], a[i] = a[i], a[k]  # backtrack


# 9. Tổng các phần tử trong mảng
def tong_mang(a, n):
    if n == 0:
        return 0
    return a[n - 1] + tong_mang(a, n - 1)


# 10. In nhị phân của số nguyên dương N
def nhi_phan(n):
    if n == 0:
        return
    nhi_phan(n // 2)
    print(n % 2, end='')

# ====================
# Ví dụ sử dụng
if __name__ == "__main__":
    n = 1234
    print("1. Tổng chữ số:", tong_chu_so(n))
    print("2. Giai thừa:", giai_thua(5))
    print("3. Fibonacci(6):", fibonacci(6))
    print("4. Lũy thừa 2^5:", luy_thua(2, 5))
    print("5. Đảo ngược số:", dao_nguoc_so(12345))
    print("6a. In tăng:")
```

```
    in_tang(5)
    print("\n6b. In giảm:")
    in_giam(5)
    print("\n7. Palindrome('level'):", is_palindrome("level"))
    print("8. Hoán vị của [1, 2, 3]:")
    hoan_vi([1, 2, 3], 0, 3)
    print("9. Tổng mảng:", tong_mang([1, 2, 3, 4], 4))
    print("10. Nhị phân của 13:", end=' ')
    nhi_phan(13)
    print()
```